

Power Apps Component Framework (PCF) Guide

CLI Reference & Step-by-Step Command Lifecycle for PCF Control Development, Packaging & Deployment

1 Component Initialization & Setup

1. Initialize PCF Component Project

```
pac pcf init --namespace kls.SearchForm --name SearchForm --template field
```

Description: Scaffolds a new Power Apps Component Framework project directory structure using the Power Platform CLI (`pac`). Sets the custom namespace (`kls.SearchForm`), control component name (`SearchForm`), and specifies the `field` control template (suitable for bound form attributes/fields).

2. Install Node Modules & UI Dependencies

```
npm install
npm install @fluentui/react
npm install react react-dom
```

Description: Downloads and restores all standard project npm packages defined in `package.json`. Additionally installs Microsoft's `@fluentui/react` framework alongside `react` and `react-dom` core libraries to enable React-based UI component rendering inside Dataverse.

2 Local Building & Workbench Testing

1. Build Control in Debug Mode

```
npm run build
```

Description: Compiles TypeScript code, bundle resources, and manifest files into unminified development assets. Includes source maps to facilitate debugging directly in browser Developer Tools.

2. Build Control in Production Mode

```
npm run build --buildMode release
```

Description: Executes a production build by compiling and minifying code assets, stripping debug overhead, and optimizing the final JavaScript package for maximum performance in Dataverse environments.

3. Start Local Test Harness / Live Preview

```
npm start
npm start watch
```

Description: Launches the local PCF test harness server in your browser to interactively test control behavior without uploading to Dataverse. Running `npm start watch` enables hot-reloading to automatically recompile and refresh upon saving code changes.

3 Solution Packaging & Build Compilation

1. Create Dataverse Solution Wrapper

```
pac solution init --publisher-name "kales Labs" --publisher-prefix kls
```

Description: Initializes a new Dataverse solution project (`.cdsproj`) in a dedicated folder. Defines publisher metadata including the publisher name (kales Labs) and custom prefix (kls) used to wrap the component into an importable solution package.

2. Reference PCF Control in Solution

```
pac solution add-reference --path C:\Repo\PCFControlRepo\src\solutions\Controls\SalesMobileDashboardControl
```

Description: Links the PCF component project path to the solution project. Ensures MSBuild bundles the built PCF component automatically when compiling the solution package.

3. Restore NuGet & Dependencies

```
msbuild /t:restore OR dotnet restore
```

Description: Fetches and restores required NuGet build tools and Power Platform SDK dependencies needed by MSBuild/dotnet to assemble the solution zip package.

4. Build PCF Solution (Development & Production)

```
# Development Build:
msbuild /t:Build OR dotnet build

# Production Release Build:
msbuild /t:Build /p:Configuration=Release OR dotnet build --configuration Release
```

Description: Compiles the project solution to produce ready-to-import Dataverse solution packages (`.zip` files) located in `bin/Release` or `bin/Debug` directory.

4 Dataverse Authentication & Direct Deployment

1. Authenticate with Power Platform Tenant

```
pac auth create
```

Description: Opens an interactive browser authentication prompt to sign in and save access credentials for your target Power Platform tenant within the PAC CLI.

2. List Connected Environments

```
pac org list
```

Description: Displays all authenticated Power Platform environments, listing their instance URLs, friendly names, active indicators, and unique Environment IDs (GUIDs).

3. Select Target Environment Context

```
pac org select --environment 718e236e-1a74-ed60-a68a-c24a467620a9
```

Description: Sets the active CLI working context to the specified Dataverse environment GUID ('718e236e-1a74-ed60-a68a-c24a467620a9'), targeting it for all subsequent deployment commands.

4. Direct Push Control to Dataverse (Fast Deployment)

```
pac pcf push --publisher-prefix kls
```

Description: Builds the PCF control and deploys it directly into the selected Dataverse environment inside a temporary developer solution. Bypasses manual solution creation and import, significantly speeding up testing and live debugging on Dataverse forms.

Pro-Tip: For rapid iterative development, use `pac pcf push` to test changes directly on your Dataverse sandbox forms. Once feature complete, compile full managed/unmanaged solutions using `dotnet build --configuration Release` inside your solution project folder for official ALM deployment across environments.